

Post-Lab Reflection (1 page max) – Taylor Series with Recursion

A.

1. The percent error decreases as N increases, because the approximation gets closer to the real value. For 60 degrees increasing N from 4 to 10 dropped the $\cos(x)$ error.
2. The largest recorded error was 100.0000% for $\cos(x)$ at 90 degrees. This high value is due to division by zero in the percent error formula, as the true value of the divider is 0. The most significant *meaningful* error was 0.007% for $\cos(x)$ at 60 degrees with $N=4$.
3. In these specific test results, $\sin(x)$ converged faster (lower error) than $\cos(x)$ when $N=4$. This is because the $\sin(x)$ series uses higher-order terms (larger factorials) which reduce the error more quickly for small N .
4. We must use radians because all calculus and Taylor series formulas are based on the assumption that the angle x is in radians. Using degrees would make the series calculation mathematically incorrect.

B.

1.

```
double cos_taylor_rec(double x, int N) {  
    int k = N - 1;  
    if (N == 0)  
        return 0.0;  
    int power = 2 * k;  
    double term_value = recursive_power(x, power) / recursive_factorial(power);  
    if (k % 2 != 0)  
        term_value *= -1.0;  
    return term_value + cos_taylor_rec(x, N - 1);  
}
```

2.

This recursive function works by counting down the total number of terms (N) we still need to add. Base Case (When Recursion Stops): The recursion stops when N is 0. It returns 0.0 because there are no more terms to add. The results from all previous calls are added to this final zero value as the function returns up the stack.

Recursive Case (What the function calls itself with): The function calls itself with $N-1$. It adds the value of the current term (term_value) to the result of the function calling itself for the rest of the terms. The N value goes down by 1 in each call.

What each recursive call represents in the Taylor series:

Each call calculates one single term of the cosine series. The term_value is the value of this current term: $(-1)^k \frac{x^{2k}}{(2k)!}$. When the function finally returns the values up the chain, it adds up all these single term values to get the final total sum of the series.

3.

Advantage: Looks Cleaner: The code is simpler and looks more like the math definition.

Disadvantage: Can be Slow and Crash: For many terms, recursion uses a lot of computer memory. This can make the program slow or even cause it to crash (a "Stack Overflow").

C.

1. The easiest part was writing the small helper functions for factorial and power. These are simple math steps with well-known rules, and I had already written both iterative and recursive versions.
2. Calculating the percent error was tricky.
3. I would make sure that the divider won't be zero, so the output wouldn't be meaningless numbers.